

Gatus, in plain terms

an uptime watchdog that pages you when the homelab breaks

Gatus is a tiny, config-file-driven **health-checking service**. You give it a list of endpoints (URLs, DNS names, pings) and the conditions that mean "healthy". It probes them on a loop, keeps a rolling history, shows a red/green dashboard at **gatus.lan**, and — the part you care about — **emails you when something goes down and again when it recovers**. It runs as one ~64 MB pod. This is the piece that turns "I'll notice when I next open Homepage" into "my phone buzzes three minutes after Immich dies."

Image ghcr.io/twin/gatus:v5.36.0

Namespace gatus (pinned to a3)

Dashboard https://gatus.lan/

Manifests clusters/home/gatus/

~25

ENDPOINTS

Cluster, Infra, Apps &
Inference probed on a loop

1m

CHECK INTERVAL

Most checks;
GPU/inference on 5m to
cut noise

~3m

TIME TO ALERT

3 consecutive failures
before an email fires

64Mi

FOOTPRINT

One pod + a 1 GB SQLite
volume for history

The core mental model (read this first)

Gatus is just a loop over three things:

- **Endpoint** — *what to hit*: an HTTP URL, a DNS query, an ICMP ping. In your setup these are the **in-cluster Service DNS names** (e.g. `http://immich-server.immich:80`), not the `.lan` URLs.
- **Conditions** — *what "healthy" means*: e.g. `[STATUS] == 200`, `[BODY].status == healthy`, `[CONNECTED] == true`. All must pass or the check is a failure.
- **Alert** — *what to do when it flips*: after N failures, send an email; after M successes, send the "resolved" email.

Why probe Service DNS instead of the pretty `.lan` URLs?

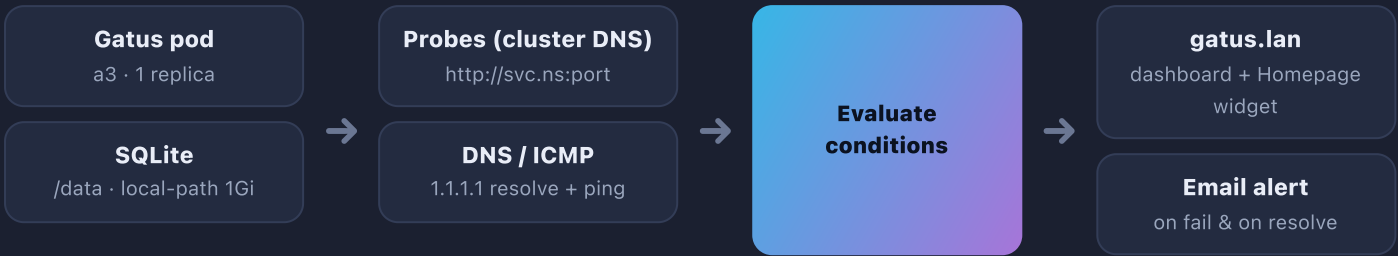
Hitting `https://immich.lan` from inside the pod would depend on **Pi-hole resolving `.lan`** and on the pod **trusting the mkcert CA** — two extra things that can break and give you *false* outages. Probing `http://immich-server.immich:80` talks straight to the app over cluster DNS, so a red marker means **the app is actually down**, not "DNS/TLS had a bad day." Traefik and Pi-hole get their *own* explicit checks so the ingress path is still covered.

What it is not

Gatus is **black-box** monitoring — "is it up, does it answer correctly, how fast." It does **not** replace Prometheus/Grafana (metrics, GPU util, trends) or Loki (logs). Think of it as the **alarm bell** that sits in front of those richer tools: Gatus tells you *something's down*; Grafana/Loki tell you *why*.

How it fits your cluster

One Deployment on **a3** (co-located with Prometheus/Grafana). It writes uptime history to a small SQLite DB on a local-path volume so history survives restarts, serves a dashboard through the usual Traefik + mkcert ingress, and Homepage renders a live up/down widget from its in-cluster API.



What's being watched (grouped on the dashboard)

GROUP	ENDPOINTS	INTERVAL	ALERTS?
Cluster	homepage, headlamp, grafana, prometheus, loki, harbor, netdata	1m / 5m	email
Infra	traefik ingress, pi-hole admin, DNS resolution (1.1.1.1), internet (ICMP), speedtest	1m / 5m	email (speedtest: no)
Apps	forgejo, vaultwarden, immich, paperless, jellyfin, navidrome, audiobookshelf, open-webui, manyfold, milvus	1m / 5m	email (media: no)
Inference	lmstudio, inference-club-agent	5m	none

GPU/inference pods cycle on and off by design, so they're **visibility-only** — shown on the dashboard but with no alert, so a deliberately-stopped model never pages you. Same reasoning for the media apps and speedtest: monitored, but a blip there isn't worth a 2 a.m. email. Flip any of these on later by adding alerts: `[{ type: email }]`.

Why a3, and why SQLite?

Storage is node-pinned local-path (per the repo's storage rules), so the pod is pinned to **a3** to keep its history volume with the rest of the monitoring stack. History is nice-to-have, not sacred — if a3 is ever rebuilt you lose the uptime graph, not any real data. Memory-only storage was the alternative (no volume, no pin) but you'd lose all history on every restart, which defeats "am I *reliably* up over the last week?"

Alerting — the part that matters

Alerting is wired but **delivery is off until you add one secret**. The dashboard and history work the moment you deploy; email only starts sending once `gatus-secrets` exists. This is deliberate — no credential is committed to this public repo, and a missing secret can't crash the pod (the `secretRef` is optional).

How an alert decides to fire

KNOB	VALUE	MEANING
failure-threshold	3	3 checks in a row must fail before the "down" email fires
success-threshold	2	2 checks in a row must pass before the "resolved" email fires
send-on-resolved	true	You get a follow-up email when it comes back — no wondering
interval	1m	So: ~3 min of real downtime → alert; ~2 min healthy → all-clear

The thresholds live in the default-alert block, so every endpoint inherits them — you only write alerts: `[{ type: email }]` per endpoint. Want a chatty endpoint to be slower to alert? Override the threshold on just that endpoint.

Turn on email (Gmail) — one-time, ~5 min

1. **Make an app password.** Google Account → Security → 2-Step Verification → **App passwords** → generate one for "Gatus". (Regular account password won't work with 2FA on; the app password is a 16-char token.)
2. **Stash it in Vaultwarden** as item `gatus-smtp` so it's recoverable and never lives on disk — matches the repo's secret convention (`scripts/vault-secret.sh`).
3. **Create the k8s secret** in the `gatus` namespace (keys are read by the config's `_${GATUS_SMTP_*}` vars):

```
# pull the app password straight from Vaultwarden into an env var (nothing written to disk)
APP_PW=$(scripts/vault-secret.sh gatus-smtp)

kubectl -n gatus create secret generic gatus-secrets \
  --from-literal=GATUS_SMTP_FROM=you@gmail.com \
  --from-literal=GATUS_SMTP_USERNAME=you@gmail.com \
  --from-literal=GATUS_SMTP_PASSWORD="$APP_PW" \
  --from-literal=GATUS_SMTP_TO=you@gmail.com

# pick up the new secret
kubectl -n gatus rollout restart deploy/gatus
```

host: `smtp.gmail.com` and port: `587` are hard-coded in the config (not secret). Only the address + app password come from the secret. Send a test by briefly scaling a watched app to zero and watching your inbox.

Better long-term: self-hosted ntfy (issue #19)

Email is the fastest path to a working phone alert with zero new services. Once you stand up **ntfy**, add an `ntfy` provider block to the config and switch each endpoint's alert type — you get instant push to your phone with no reliance on Gmail SMTP, and nothing leaves the LAN. Gatus supports both; you can even run them side by side during the switch.

Operating it day-to-day

Deploy / first run

```
# 1. mint the TLS cert (gatus.lan already added to the script)
bash scripts/lan-certs.sh

# 2. apply the stack
kubectl apply -k clusters/home/gatus

# 3. (optional) enable email – see page 3
```

Add or change an endpoint

Everything lives in `clusters/home/gatus/configmap.yaml`. Add a block, then roll the pod (a plain ConfigMap doesn't auto-restart the Deployment):

```
- name: my-service
  group: Apps
  url: "http://my-svc.my-ns:80/health"
  interval: 1m
  conditions: ["[STATUS] == 200"]
  alerts: [{ type: email }]      # omit this line = dashboard-only, no page

# then:
kubectl -n gatus rollout restart deploy/gatus
```

Condition placeholders you'll actually use

PLACEHOLDER	EXAMPLE CONDITION	USE IT FOR
[STATUS]	[STATUS] == 200 · [STATUS] < 400	HTTP health
[BODY]	[BODY].status == healthy	JSON health payloads (e.g. Harbor)
[RESPONSE_TIME]	[RESPONSE_TIME] < 500	Latency SLOs (ms)
[CONNECTED]	[CONNECTED] == true	ICMP ping / TCP reachability
[DNS_RCODE]	[DNS_RCODE] == NOERROR	DNS resolution checks
[CERTIFICATE_EXPIRATION]	[CERTIFICATE_EXPIRATION] > 168h	Warn before a TLS cert expires

Troubleshooting

SYMPTOM	LIKELY CAUSE / FIX
Endpoint red but the app works	Wrong service name/port or health path. Check with <code>kubectl get svc -n <ns></code> ; test from inside: <code>kubectl -n gatus exec deploy/gatus -- wget -q0- <url></code> .
No emails despite a real outage	Secret missing/typo'd. <code>kubectl -n gatus logs deploy/gatus grep -i alert</code> — look for "provider not configured". Verify the Gmail app password, not the login password.
Pod won't start	Config YAML error. <code>kubectl -n gatus logs deploy/gatus</code> shows the parse line. Validate with <code>kubectl kustomize clusters/home/gatus</code> before applying.
Homepage widget empty	Widget reads <code>http://gatus.gatus:8080</code> — confirm the Service and that Homepage re-discovered the ingress.

Where Gatus sits in the bigger reliability picture

Gatus answers "is it up?" instantly and loudly. It's the front line of a layered plan — the other layers are already filed as tickets in `brian/home-lab`:

- 1 Gatus — detection** done (this doc)
Know within ~3 min that something's down, and get told when it recovers. The alarm bell.
- 2 ntfy — better delivery** #19
Swap email for self-hosted push so alerts never leave the LAN and hit your phone instantly. Gatus is already ntfy-ready.
- 3 Alertmanager + Karma — metric alerts** #24
Gatus catches "down"; Prometheus alert rules catch "degrading" (disk filling, GPU too hot, memory creeping). Complementary, not overlapping.
- 4 k8up / Velero — recovery** #18 · #21
Detection is only half the job. When Gatus says Immich is down because a disk died, backups are what get it back. `local-path` has no redundancy today.

Bottom line

You now have a **single pane that goes red the instant any core service stops answering**, backed by ~25 checks across the cluster, ingress, DNS, and the internet uplink — for the cost of one 64 MB pod. Deploy it, spend five minutes wiring the Gmail app password, and the homelab goes from "I hope it's fine" to "it will tell me if it isn't." Then graduate to ntfy (#19) when you want the pushes to stay fully in-house.

Quick reference

THING	VALUE
Dashboard	<code>https://gatus.lan/</code>
Manifests	<code>clusters/home/gatus/</code> — apply with <code>kubectl apply -k</code>
Config to edit	<code>clusters/home/gatus/configmap.yaml</code> → <code>rollout restart deploy/gatus</code>
Enable email	create secret <code>gatus-secrets</code> (4 <code>GATUS_SMTP_*</code> keys)
Node / storage	a3 · SQLite on <code>local-path</code> (1Gi)
Logs	<code>kubectl -n gatus logs deploy/gatus</code>